



KTU NOTES

The learning companion.

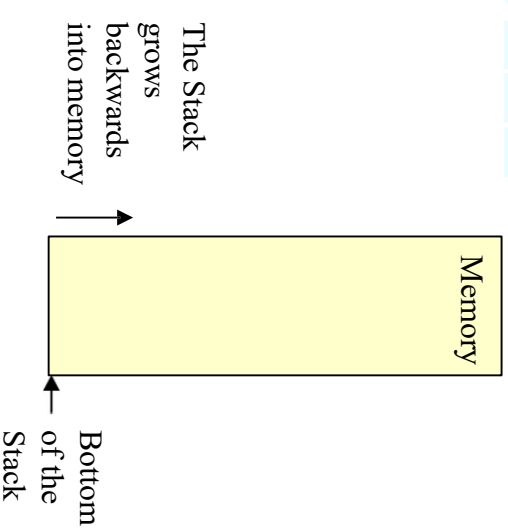
**KTU STUDY MATERIALS | SYLLABUS | LIVE
NOTIFICATIONS | SOLVED QUESTION PAPERS**

Website: www.ktunotes.in

Stack and Subroutines

The Stack

- **The stack is an area of memory identified by the programmer for temporary storage of information.**
- The stack is a LIFO structure.
 - Last In First Out.
- The stack normally grows backwards into memory.
 - In other words, the programmer defines the bottom of the stack and the stack grows up into reducing address range.



The Stack

- Given that the stack grows backwards into memory, it is customary to place the bottom of the stack at the end of memory to keep it as far away from user programs as possible.
 - In the 8085, the stack is defined by setting the SP (Stack Pointer) register.
 - **LXI SP, FFFFH**
 - This sets the Stack Pointer to location FFFFH (end of memory for the 8085).
 - The Size of the stack is limited only by the available memory
-

The Stack

- Only two operations possible on Stack
- **PUSH**
- **POP**

Ktunotes.in

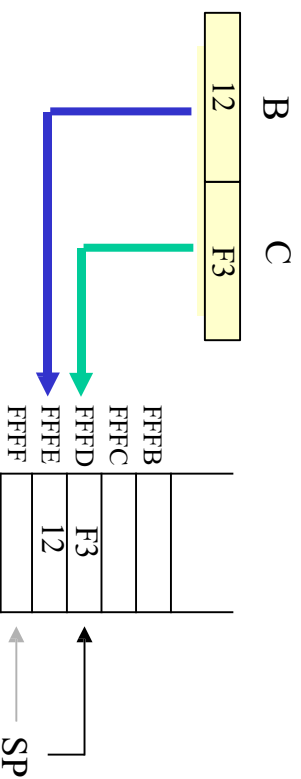


Saving Information on the Stack

- Information is saved on the stack by **PUSHing** it on.
 - It is retrieved from the stack by **POPing** it off.
 - The 8085 provides two instructions: **PUSH** and **POP** for storing information on the stack and retrieving it back.
 - Both PUSH and POP work with register pairs ONLY.
-

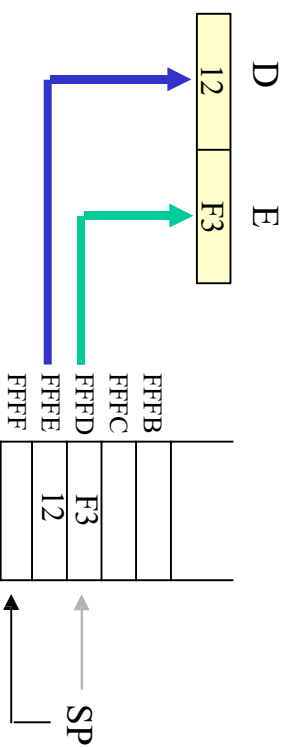
The PUSH Instruction

- **PUSH B (1 Byte Instruction) Rp**
 - Decrement SP
 - Copy the contents of register B to the memory location pointed to by SP
 - Decrement SP
 - Copy the contents of register C to the memory location pointed to by SP



The POP Instruction

- POP D (1 Byte Instruction)
 - Copy the contents of the memory location pointed to by the SP to register E
 - Increment SP
 - Copy the contents of the memory location pointed to by the SP to register D
 - Increment SP



Operation of the Stack

- During pushing, the stack operates in a “**decrement then store**” style.
 - The stack pointer is decremented first, then the information is placed on the stack.
 - During popping, the stack operates in a “**use then increment**” style.
 - The information is retrieved from the top of the the stack and then the pointer is incremented.
 - **The SP pointer always points to “the top of the stack”.**
-

LIFO

- The order of PUSHs and POPs must be opposite of each other in order to retrieve information back into its original location.

ALP TO EXCHANGE THE CONTENTS OF BC AND DE

LXI SP,8501H

PUSH B

PUSH D

POP D

POP B

Ktunotes.in

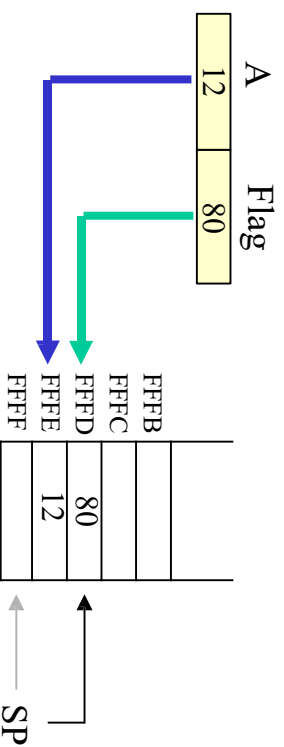
- Reversing the order of the POP instructions will result in the exchange of the contents of BC and DE.
-

The PSW Register Pair

- The 8085 recognizes one additional register pair called the PSW (Program Status Word).
 - This register pair is made up of the Accumulator and the Flags registers.
 - It is possible to push the PSW onto the stack, do whatever operations are needed, then POP it off of the stack.
 - The result is that the contents of the Accumulator and the status of the Flags are returned to what they were before the operations were executed.
-

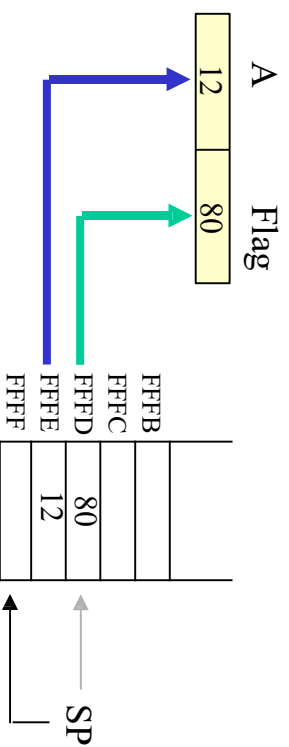
PUSH PSW Register Pair

- PUSH PSW (1 Byte Instruction)
 - Decrement SP
 - Copy the contents of register *A* to the memory location pointed to by SP
 - Decrement SP
 - Copy the contents of Flag register to the memory location pointed to by SP



Pop PSW Register Pair

- POP PSW (1 Byte Instruction)
 - Copy the contents of the memory location pointed to by the SP to Flag register
 - Increment SP
 - Copy the contents of the memory location pointed to by the SP to register A
 - Increment SP



Ktunotes.in

Subroutines

Subroutines

- A subroutine is a group of instructions that will be used repeatedly in different locations of the program.
 - Rather than repeat the same instructions several times, they can be grouped into a subroutine that is called from the different locations.
 - In Assembly language, a subroutine can exist anywhere in the code.
 - However, it is customary to place subroutines separately from the main program.
-

Subroutines

- The 8085 has two instructions for dealing with subroutines.
 - The CALL instruction is used to redirect program execution to the subroutine.
 - The RET instruction is used to return the execution to the calling routine.

CALL-RET

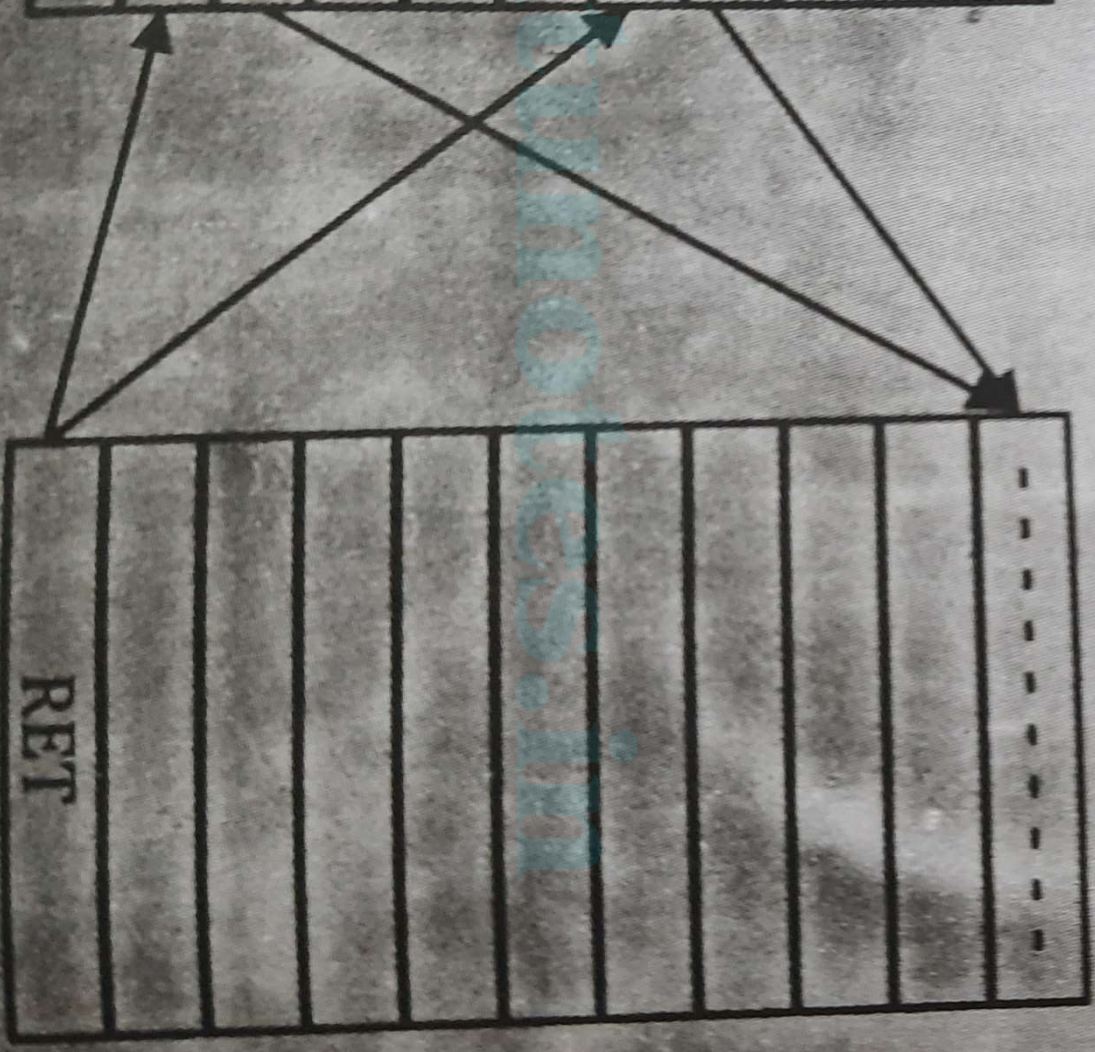
- CALL-RET instructions are used in subroutine operations.
 - On getting a CALL in the main program, the current program counter content is pushed into the stack and loads the PC with the first memory location of the subroutine.
 - When RET instruction is executed the memory address stored on the stack is retrieved and sequence of execution is retrned to main program
-

Main Program

- - - - -
- - - - -
- - - - -
- - - - -
CALL subroutine
- - - - -
- - - - -
- - - - -
- - - - -
- - - - -
- - - - -
- - - - -

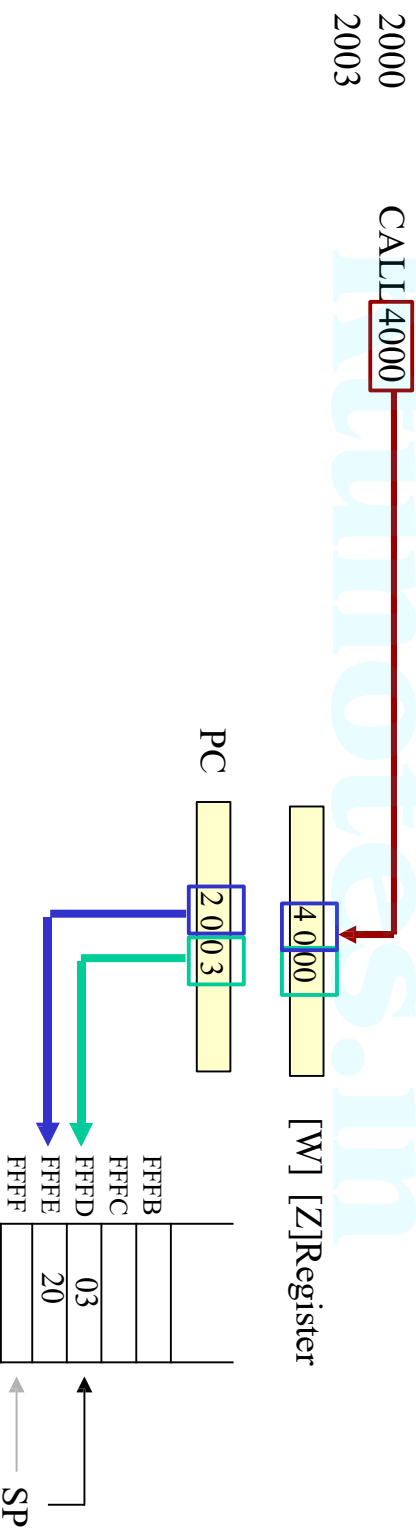
Subroutine

- - - - -
RET



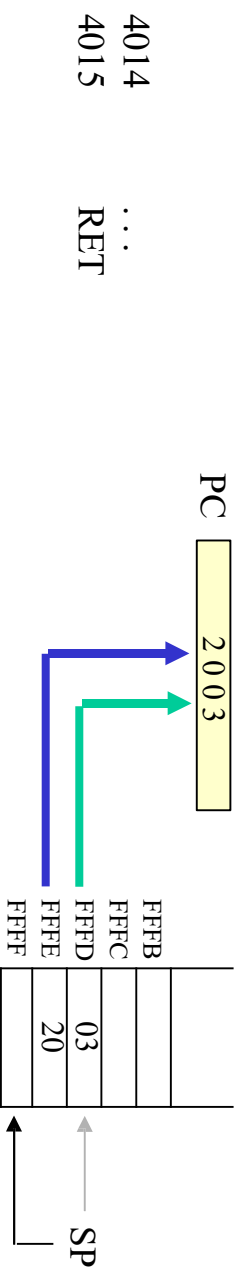
The CALL Instruction

- CALL 4000H (3 byte instruction)
 - When CALL instruction is fetched, the MP knows that the next two Memory location contains 16bit subroutine address in the memory.



The RET Instruction

- RET (1 byte instruction)
 - Retrieve the return address from the top of the stack
 - Load the program counter with the return address.



The execution of CALL and RET instructions can best be explained with an example.

Example: Assume user available memory ranges from 2000H to 23FFH

Memory Address

2000	LXI	SP, 2400 H	; Initialization of SP
2040	CALL	2070 H	; Call subroutine at 2070 H
2041			
2042			
2043			; address of next instruction after CALL instruction

ktunotes.in

205F	HLT	; End of main program
2070	SUBROUTINE 1	; beginning of subroutine
:	:	
:	:	
207F	RET	; end of subroutine
2080		
:		
:		
23FF		
2400		

2400
can be made.

Label			
	LXI		SP, XX50H
	MVI		A, 00H
Repeat	OUT		PORT#
	MVI		B, 0AH
	<u>CALL</u>		<u>DELAY</u>
	MVI		A, FFH
	OUT		PORT#
	MVI		B, 05H
	CALL		DELAY
	JMP		Repeat

Delay Subroutine:

Label	Mnemonics	Operand
DELAY	PUSH	PSW
Second	LXI	H, Count
Loop	DCX	H
	MOV	A, H
	ORA	L
	INZ	Loop
	DCR	B
	INZ	Second
	POP	PSW
	RET	

Conditional CALL and RTE Instructions²³

- The 8085 supports conditional CALL and conditional RTE instructions.
 - The same conditions used with conditional JUMP instructions can be used.
 - CC, call subroutine if Carry flag is set.
 - CNC, call subroutine if Carry flag is not set
 - RC, return from subroutine if Carry flag is set
 - RNC, return from subroutine if Carry flag is not set
 - RZ, If zero flag is set Z=1
 - Etc.
-

DELAY PROGRAMS

Hyunnotes.in

Time delay using one register

Label	Opcode	Operand	Comments	T-states
	MVI	C,FFH	:Load register C	7
LOOP:	DCR	C	:Decrement C	4
	JNZ	LOOP	:Jump back to ; decrement C	10/7

Time delay using a register pair

Label	Opcode	Operand	Comments	T-states
LOOP:	LXI	B,2384H	:Load BC with 16-bit count	10
	DCX	B	:Decrement (BC) by one	6
	MOV	A,C	:Place contents of C in A	4
	ORA	B	:OR (B) with (C) to set Zero flag	4
	JNZ	LOOP	:If result $\neq$ 0, jump back to LOOP	10/7

Time delay using a loop within a loop technique

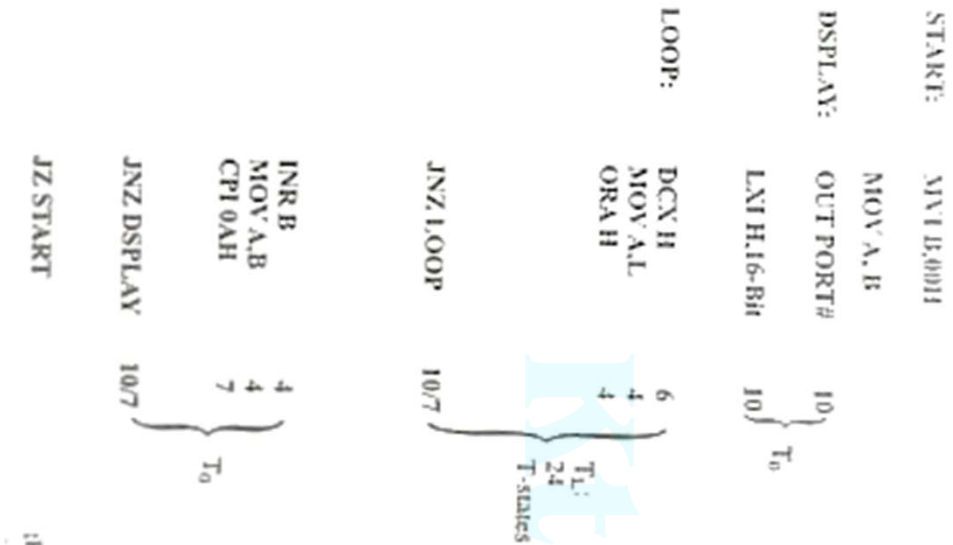
MVI B,38H	7T
LOOP2: MVI C,FFH	7T
LOOP1: DCR C	4T
JNZ LOOP1	10/7T
DCR B	4T
JNZ LOOP2	10/7T

Hexadecimal counter(FF TO 00H) with 1ms time delay between counts(ASSUME CLOCK OF 2MHz)

Label	Mnemonics
	MVI B,00H
NEXT:	DCR B
	MVI C,COUNT
DELAY:	DCR C
	JNZ DELAY
	MOV A,B
	OUT PORT#
	JMP NEXT

Zero to nine counter with 1s time delay between counts

(ASSUME CLOCK OF 1 MHz)



generate a continuous square wave with a period of $500\mu\text{s}$, clock period is 325ns and use bit D_0 to output the square wave

```
MVI D,AA  
E:  MOV A,D  
    RLC
```

```
    MOV D,A  
    ANI 01H
```

```
    OUT PORT1
```

```
    MVI B,COUNT (7T)
```

```
Y:   DCR B (4T)
```

```
    JNZ DELAY (10/7T)
```

```
    JMP ROTATE (10T)
```

Ktunnotes.in

DELAY SUBROUTINE

```
      MVI B, FFH  
LOOP: DCR B  
      JNZ LOOP  
      RET
```

Ktunotes.in